# Lab Assignment 20.2

| | |
|---|---|
| **Program** | **: B. Tech (CSE)** |
| **Course Title** | **: AI Assisted Coding** |
| **Course Code** | **: 23CS002PC304** |
| **Semester** | **: II** |
| **Academic Session** | **: 2025-2026** |
| **Name of Student** | **: Sruthi** |
| **Enrollment No.** | **: 2403A51L10** |
| **Batch No.** | **: 51** |
| **Date** | **: 10-04-2026** |

## Task 1:

**Prompt 1:** Generate a simple Python program for user registration and login that stores usernames and passwords in a dictionary.

## Code:

```python
def register_user(users):
    username = input("Enter a username: ")
    if username in users:
        print("Username already exists. Please choose a different username.")
        return
    password = input("Enter a password: ")
    users[username] = password
    print("Registration successful!")
def login_user(users):
    username = input("Enter your username: ")
    password = input("Enter your password: ")
    if username in users and users[username] == password:
        print("Login successful!")
    else:
        print("Invalid username or password.")
# Main program
users = {}
while True:
    choice = input("Do you want to register (r) or login (l)? (Enter 'q' to quit): ")
    if choice == 'r':
        register_user(users)
    elif choice == 'l':
        login_user(users)
    elif choice == 'q':
        print("Exiting the program.")
        break
    else:
        print("Invalid choice. Please enter 'r', 'l', or 'q'.")
```

**Output:**

```
Do you want to register (r) or login (l)? (Enter 'q' to quit): r
Enter a username: Sweety
Enter a password: Sweety
Registration successful!
Do you want to register (r) or login (l)? (Enter 'q' to quit): r
Enter a username: Sweety
Enter a password: Sweety
Registration successful!
Do you want to register (r) or login (l)? (Enter 'q' to quit): l
Enter your username: Sweety
Do you want to register (r) or login (l)? (Enter 'q' to quit): l
Enter your username: Sweety
Enter your username: Sweety
Enter your password: Sweety
Enter your password: Sweety
Login successful!
Do you want to register (r) or login (l)? (Enter 'q' to quit): q
Do you want to register (r) or login (l)? (Enter 'q' to quit): q
Exiting the program.
```

**Prompt 2:** Refactor the program to store passwords securely using hashing (use hashlib or bcrypt). Ensure login validation still works correctly.

**Code:**

```python
import hashlib
def hash_password(password):
    return
    hashlib.sha256(password.encode()).hexdigest()
def register_user(users):
    username = input("Enter a username: ")
    if username in users:
        print("Username already exists. Please choose a different username.")
        return
    password = input("Enter a password: ")
    users[username] = hash_password(password)
    print("Registration successful!")
def login_user(users):
    username = input("Enter your username: ")
    password = input("Enter your password: ")
    if username in users and users[username] == hash_password(password):
        print("Login successful!")
    else:
        print("Invalid username or password.")
# Main program
users = {}
while True:
    choice = input("Do you want to register (r) or login (l)? (Enter 'q' to quit): ")
    if choice == 'r':
        register_user(users)
    elif choice == 'l':
        login_user(users)
    elif choice == 'q':
        print("Exiting the program.")
        break
    else:
        print("Invalid choice. Please enter 'r', 'l', or 'q'.")
```

**Output:**

```
Do you want to register (r) or login (l)? (Enter 'q' to quit): r
Enter a username: Sweety
Enter a password: Sweety
Registration successful!
Do you want to register (r) or login (l)? (Enter 'q' to quit): l
Enter your username: Sweety
Enter your password: Sweety
Login successful!
Do you want to register (r) or login (l)? (Enter 'q' to quit): q
Exiting the program.
```

**Explanation:**

1. The original code stored passwords in plain text, which is a security risk. The refactored code uses the hashlib library to hash passwords before storing them in the dictionary.
2. The login validation is updated to compare the hashed version of the input password with the stored hashed password, ensuring that the login process still works correctly while enhancing security.

## Task 2:

**Prompt1:** Generate a Python script that fetches data from an API using a hardcoded API key.

**Code:**

```python
import requests
def fetch_data():
    url = "https://jsonplaceholder.typicode.com/posts"
    try:
        response = requests.get(url)
        response.raise_for_status()  # Check if the request was successful
        data = response.json()
        print("Data fetched successfully:")
        print(data[:2])
    except requests.exceptions.RequestException as e:
        print(f"An error occurred: {e}")
# Test the function
fetch_data()
```

**Output:**

```
Data fetched successfully:
[{'userId': 1, 'id': 1, 'title': 'sunt aut facere repellat provident occaecati excepturi optio reprehenderit', 'body': 'quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas
totam\nnostrum rerum est autem sunt rem eveniet architecto'}, {'userId': 1, 'id': 2, 'title': 'qui est esse', 'body': 'est rerum tempore vitae\nsequi sint nihil reprehenderit dolor beatae ea dolores neque\nfugiat blanditiis
 voluptate porro vel nihil molestiae ut reiciendis\nqui aperiam non debitis possimus qui neque nisi nulla'}]
```

**Prompt2:** Modify the script to securely access the API key using environment variables instead of hardcoding it.

**Code:**

```python
import os
import requests
def fetch_data():
    api_key = os.getenv("API_KEY")
    if not api_key:
        print("API key not found. Please set the API_KEY environment variable.")
        return
    url = "https://jsonplaceholder.typicode.com/posts"
    headers = {"Authorization": f"Bearer {api_key}"}
    try:
        response = requests.get(url, headers=headers)
        response.raise_for_status()  # Check if the request was successful
        data = response.json()
        print("Data fetched successfully:")
        print(data[:2])
    except requests.exceptions.RequestException as e:
        print(f"An error occurred: {e}")
# Test the function
fetch_data()
```

**Output:**

```
Data fetched successfully:
[{'userId': 1, 'id': 1, 'title': 'sunt aut facere repellat provident occaecati excepturi optio reprehenderit', 'body': 'quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas
totam\nnostrum rerum est autem sunt rem eveniet architecto'}, {'userId': 1, 'id': 2, 'title': 'qui est esse', 'body': 'est rerum tempore vitae\nsequi sint nihil reprehenderit dolor beatae ea dolores neque\nfugiat blanditii
 voluptate porro vel nihil molestiae ut reiciendis\nqui aperiam non debitis possimus qui neque nisi nulla'}]
```

**Explanation:**

The original code had a hardcoded API key, which is a security risk. The modified code retrieves the API key from an environment variable, which is a more secure practice.

## Task 3:

**Prompt 1:** Generate a simple Python program that encrypts and decrypts text using a basic method like Base64 encoding.

**Code:**

```python
import base64
def encrypt_text(plain_text):
    encoded_bytes = base64.b64encode(plain_text.encode("utf-8"))
    encoded_str = encoded_bytes.decode("utf-8")
    return encoded_str
def decrypt_text(encoded_str):
    decoded_bytes = base64.b64decode(encoded_str.encode("utf-8"))
    decoded_str = decoded_bytes.decode("utf-8")
    return decoded_str
# Test the functions
original_text = "Hello, World!"
encrypted_text = encrypt_text(original_text)
print(f"Encrypted Text: {encrypted_text}")
decrypted_text = decrypt_text(encrypted_text)
print(f"Decrypted Text: {decrypted_text}")
```

**Output:**

```
Encrypted Text: SGVsbG8sIFdvcmxkIQ==
Decrypted Text: Hello, World!
```

**Prompt 2:** Rewrite the program using a strong encryption method in Python (such as cryptography library with Fernet).

**Code:**

```python
from cryptography.fernet import Fernet
def generate_key():
    return Fernet.generate_key()
def encrypt_text(plain_text, key):
    fernet = Fernet(key)
    encrypted_text = fernet.encrypt(plain_text.encode())
    return encrypted_text
def decrypt_text(encrypted_text, key):
    fernet = Fernet(key)
    decrypted_text = fernet.decrypt(encrypted_text).decode()
    return decrypted_text
# Test the functions
key = generate_key()
original_text = "Hello, World!"
encrypted_text = encrypt_text(original_text, key)
print(f"Encrypted Text: {encrypted_text}")
decrypted_text = decrypt_text(encrypted_text, key)
print(f"Decrypted Text: {decrypted_text}")
```

**Output:**

```
Encrypted Text: b'gAAAAABp4HX3NIhwGxDL83dNArdRS_vw4NQ1SzZtp_dEmb--9j8-xpifDyZg_VMJ92L5qlvRs1jNq_V94kC6rHkWHgUAGeZSQw=='
Decrypted Text: Hello, World!
```

**Explanation:**

1. The original code used Base64 encoding, which is not a secure encryption method. The rewritten code uses the Fernet class from the cryptography library, which provides strong encryption.
2. The original code did not require a key for encryption and decryption, while the rewritten code generates a key and uses it for both encryption and decryption, ensuring that the data is securely encrypted and can only be decrypted with the correct key.

## Task 4:

**Prompt 1:** Generate a simple Flask web application with a login API that checks username and password.

**Code:**

```python
from flask import Flask, request, jsonify
app = Flask(__name__)
users = {
    "user1": "password1",
    "user2": "password2"
}
@app.route('/login', methods=['POST'])
def login():
    data = request.get_json()
    username = data.get('username')
    password = data.get('password')
    if username in users and users[username] == password:
        return jsonify({"message": "Login successful!"}), 200
    else:
        return jsonify({"message": "Invalid username or password."}), 401
if __name__ == '__main__':
    app.run(debug=True)
```

**Output:**

POST http://127.0.0.1:5000/login

Params Authorization Headers (9) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
{
  "username": "user1",
  "password": "password1"
}
```

Body Cookies Headers (5) Test Results Status: 20

Pretty Raw Preview JSON

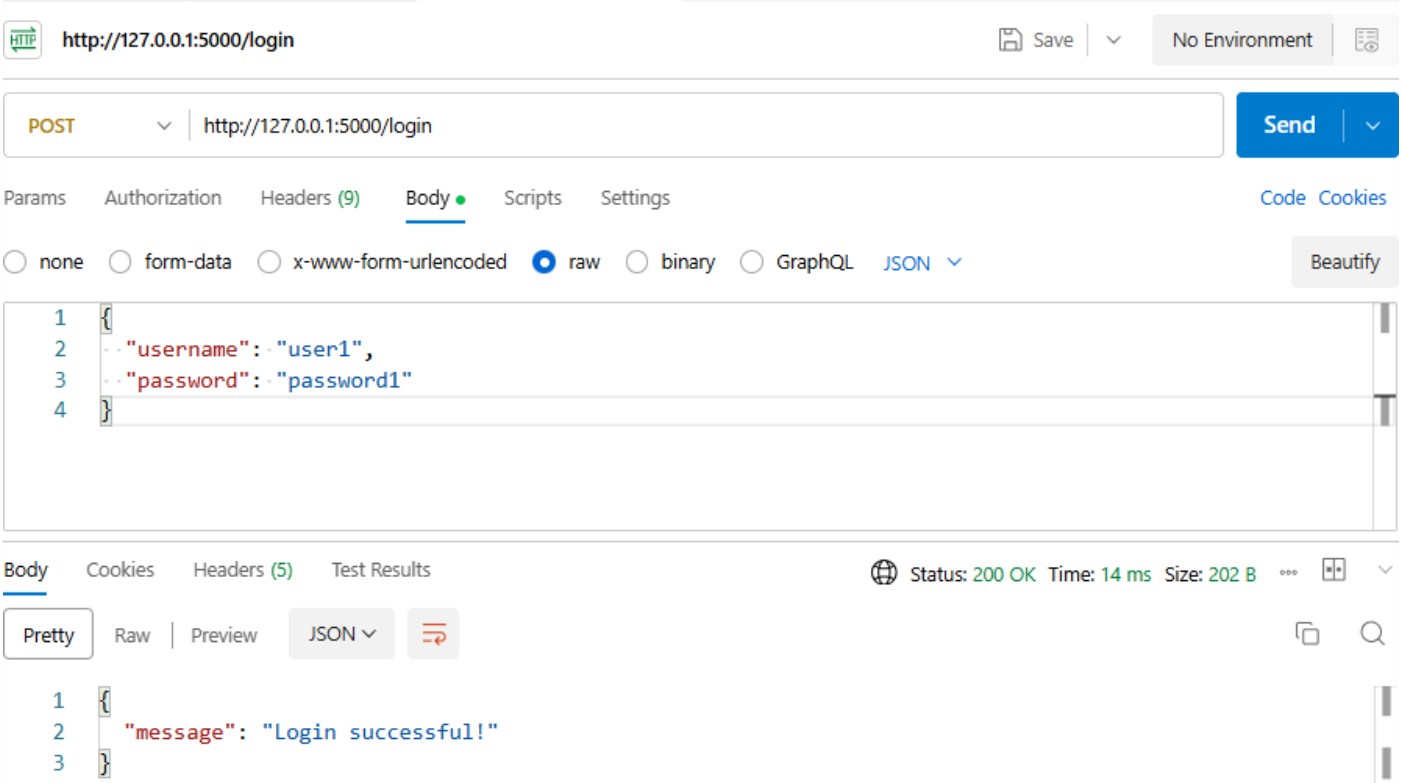
```
{
  "message": "Login successful!"
}
```

**Prompt 2:** Refactor the Flask application to improve security by adding password hashing, input validation, and better authentication practices.

**Code:**

```python
from flask import Flask, request, jsonify
import hashlib
app = Flask(__name__)
users = {
    "user1": hashlib.sha256("password1".encode()).hexdigest(),
    "user2": hashlib.sha256("password2".encode()).hexdigest()
}
def hash_password(password):
    return hashlib.sha256(password.encode()).hexdigest()
@app.route('/login', methods=['POST'])
def login():
    data = request.get_json()
    username = data.get('username')
    password = data.get('password')
    if not username or not password:
        return jsonify({"message": "Username and password are required."}), 400
    if username in users and users[username] == hash_password(password):
        return jsonify({"message": "Login successful!"}), 200
    else:
        return jsonify({"message": "Invalid username or password."}), 401
if __name__ == '__main__':
    app.run(debug=True)
```

## Output:




## Explanation:

- Instead of storing plain text passwords, the program now hashes passwords using SHA-256 before storing them in the user's dictionary. This adds a layer of security, as even if the data is compromised, the actual passwords are not exposed.
- The login endpoint now checks if both username and password are provided in the request. If either is missing, it returns a 400 Bad Request response with an appropriate message. This prevents potential issues with missing data and improves the robustness of the API.

# Task 5:

**Prompt 1:** Generate a Python program that takes two numbers as input and performs division.

## Code:

```python
def divide_numbers(num1, num2):
    if num2 == 0:
        return "Error: Division by zero is not allowed."
    else:
        return num1 / num2
# Test the function
number1 = float(input("Enter the first number: "))
number2 = float(input("Enter the second number: "))
result = divide_numbers(number1, number2)
print(f"The result of dividing {number1} by {number2} is: {result}")
```

## Output:

```
Enter the first number: 10
Enter the second number: 5
The result of dividing 10.0 by 5.0 is: 2.0
```

**Prompt 2:** Modify the program to include proper exception handling using try-except blocks and display user-friendly error messages.

**Code:**

```
def divide_numbers(num1, num2):
    try:
        result = num1 / num2
        return result
    except ZeroDivisionError:
        return "Error: Division by zero is not allowed."
    except Exception as e:
        return f"An unexpected error occurred: {e}"
# Test the function
try:
    number1 = float(input("Enter the first number: "))
    number2 = float(input("Enter the second number: "))
    result = divide_numbers(number1, number2)
    print(f"The result of dividing {number1} by {number2} is: {result}")
except ValueError:
    print("Error: Please enter valid numbers.")
```

**Output:**

```
Enter the first number: 10
Enter the second number: 5
The result of dividing 10.0 by 5.0 is: 2.0
```

**Explanation:**

In the original code, there was no exception handling for division by zero or invalid input. The modified code includes a try-except block to catch a ZeroDivisionError when the second number is zero and a ValueError when the user inputs non-numeric values. This ensures that the program does not crash and provides user-friendly error messages instead. Additionally, a general exception handler is included to catch any other unexpected errors that may occur during execution.